Policy Rule Engine + Rule Builder Framework

for AI/workflow governance.

This is exactly how enterprise governance platforms evolve:

- ServiceNow GRC
- Microsoft Purview
- Prisma Cloud
- Wiz
- Palo Alto AI Runtime Security
- Azure Policy
- AWS Config Rules

They all separate:

1. Policy
2. Rule
3. Binding
4. Enforcement
5. Scope
6. Actions

You are now building that architecture layer.

Recommended Governance Object Model

You should separate the system into:

Object	Purpose
Workflow	The governed asset
Policy	A governance package
Rule	Individual evaluation logic

Object	Purpose
Binding	Attachment between policy and target
Violation	Runtime finding
Action	Response behavior

Recommended Governance Page Model

Governance Page

Policy Version Status Workflows Attached Risk Actions

Policies become reusable governance templates.

Example:

Policy	Workflows
AI Safety Controls	12 workflows
External AI Governance	8 workflows
Production Restrictions	24 workflows

Workflow Page

Separate page:

Workflow Policies Applied Risk Score Violations

Example:
| Report Publisher E-Mail | AI Safety Controls | 95 | 4 |

This separation is architecturally correct.

Your Next Major Architectural Decision

You need:

RULE TYPES

Instead of every rule being fully custom.

This is critical.

Recommended Architecture

Policy

Container object.

```
{
 "id": "policy-001",
 "name": "AI Safety Controls",
 "version": "v1",
 "status": "active"
}
```

Rules

Typed governance logic.

```
{
 "id": "rule-001",
 "policy_id": "policy-001",
 "type": "pii_detection"
}
```

Why Rule Types Matter

Because the SPA can dynamically render:

- fields
- dropdowns
- validation
- helper UIs
- JSON editors
- logic builders

based on:

rule.type

This is exactly how enterprise policy systems work.

Recommended Rule Type System

You should define a fixed catalog:

Rule Type	Purpose
pii_detection	Detect sensitive data
ai_provider_detection	Detect external AI
prompt_inspection	Inspect prompts
logging_restriction	Restrict logging
data_classification	Require tags/classification
approval_requirement	Human approval
encryption_enforcement	HTTPS/TLS enforcement
token_limit	LLM token governance
model_allowlist	Approved AI providers
workflow_scope	Restrict environments
execution_rate	Prevent runaway automation
outbound_domain	Restrict outbound calls

This becomes:

- scalable
- extensible
- maintainable

CRITICAL DESIGN PRINCIPLE

Separate:

Rule Type

from

Rule Configuration

Example:

Rule Type Definition

```
{
  "type": "pii_detection",
  "display_name": "PII Detection",
  "description": "Detect sensitive fields in workflows"
}
```

Rule Instance

```
{
  "rule_type": "pii_detection",
  "config": {
    "inspection_targets": [
      "http_body",
      "prompt",
      "email"
    ],
    "fields": [
      "ssn",
      "dob",
      "credit_card"
    ],
    "severity": "high",
    "actions": [
      "flag_workflow",
      "generate_event"
    ]
```

```
 }
}
```

THIS IS THE KEY

Now your SPA can render fields dynamically.

Recommended SPA Rule Builder

Step 1 — Select Rule Type

Dropdown:

```
Select Rule Type
----------------
PII Detection
External AI Detection
Prompt Inspection
Logging Restriction
Encryption Enforcement
Human Approval
```

Step 2 — Dynamic Rule Form

The form changes based on:

rule_type

Example:

Rule Type = PII Detection

Render:

Field	UI Type
Rule Name	Text
Description	Textarea

Field	UI Type
Inspection Targets	Multi-select
Sensitive Fields	JSON helper
Severity	Dropdown
Actions	Multi-select

JSON Helper

Very important.

You do NOT want raw JSON editing initially.

Use:

- token chips
- add/remove rows
- import/export JSON

Example UI:

Field Name
ssn
dob
credit_card

[+ Add Field]

Internally stored as:

```
[
  "ssn",
  "dob",
  "credit_card"
]
```

Recommended Standard UI Components

Type	Component
string	text input
description	textarea
enum	dropdown
multi_enum	multi-select
json_array	chip editor
logic_builder	AND/OR UI
action_selector	action chips
severity	colored dropdown
scope_selector	workflow/tag/env selector

Your Rules Become Metadata-Driven

This is the major breakthrough.

Instead of hardcoding forms:

You create:

RULE SCHEMAS

Example Rule Schema

```
{
 "type": "pii_detection",
 "label": "PII Detection",
 "fields": [
  {
   "name": "rule_name",
   "type": "string",
   "required": true
```

```
  },
  {
   "name": "inspection_targets",
   "type": "multi_enum",
   "options": [
    "webhook_payload",
    "http_body",
    "prompt",
    "email"
   ]
  },
  {
   "name": "fields",
   "type": "json_array"
  },
  {
   "name": "severity",
   "type": "enum",
   "options": [
    "low",
    "medium",
    "high",
    "critical"
   ]
  }
 ]
}
```

THIS CHANGES EVERYTHING

Now your SPA can:

- auto-render forms
- auto-validate
- auto-version
- auto-document

- dynamically evolve

without rewriting UI code.

Recommended Rule Evaluation Model

Each rule should eventually compile into:

```
{
 "when": {
  "contains_pii": true,
  "external_ai": true
 },
 "then": {
  "severity": "critical",
  "actions": [
   "block_execution"
  ]
 }
}
```

This becomes your:

- runtime governance engine
- policy evaluation engine

Strong Architectural Recommendation

You should define:

1. Rule Type Registry

Backend catalog.

---

2. Rule Schema Engine

Dynamic SPA generation.

---

3. Policy Binding Engine

Attach policies to:

- workflows
- tags
- environments
- providers

---

4. Runtime Evaluation Engine

Evaluates workflows against rules.

---

5. Violation Engine

Stores:

- findings
- timestamps
- remediation
- audit trail

Recommended Database Model

---

policies

id
name
description
version
status
created_at

---

rules

id
policy_id
rule_type
config_json
severity
enabled

---

rule_schemas

type
display_name
schema_json
ui_schema_json

---

policy_bindings

id
policy_id
binding_type
target_id

---

violations

id
workflow_id
rule_id
severity
message
status
detected_at

Final Architectural Insight

You are no longer building:

“workflow settings.”

You are building:

A Governance Policy Orchestration Platform